

# 第5回：検索してから回答する

キーワード検索でRAGの流れを学ぶ

# 今日のゴールと流れ

---

検索と生成を別の処理として説明する。

| 時間  | 内容           |
|-----|--------------|
| 10分 | 資料全体を渡す方法の限界 |
| 20分 | RAG、断片、出典の設計 |
| 20分 | キーワード検索のコード  |
| 30分 | 検索の実装と失敗例の観察 |
| 10分 | 振り返り・提出      |

## RAGとは

---

Retrieval-Augmented Generation：検索拡張生成。

質問に関係する情報を検索し、その情報を入力に加えて生成します。

この教材では、登録した資料の中を検索します。

RAGはインターネット検索だけを指す言葉ではありません。

ベクトル検索を使わなくても、検索して生成する構成を作れます。

## 準備と質問時の処理

---

| 準備するとき     | 質問を受けたとき     |
|------------|--------------|
| 資料を集める     | 質問と検索語を受け取る  |
| 見出しごとに分割する | 関連する断片を探す    |
| 出典と本文を保存する | 選んだ断片をLLMへ渡す |
| 内容を点検する    | 回答と参照資料を表示する |

準備は資料更新時、検索と生成は質問ごとに行います。

## チャンク：資料の断片

---

教材では `##` の見出しごとに分けます。

- 小さすぎると、主語や条件が失われる
- 大きすぎると、無関係な内容も一緒に選ばれる
- 見出し、ファイル名、本文を一緒に保存する
- 表や手順は、意味が途切れない区切りを考える

1000文字を超える見出しは、教材の読み込み処理がエラーにします。

## 検索語と質問

---

| 入力欄 | 例                  |
|-----|--------------------|
| 質問  | SQLiteにデータを保存するには？ |
| 検索語 | SQLite 保存          |

初学者向けに、日本語の単語分割は人が行います。  
空白で区切った各単語が含まれるかを調べます。  
自然文のまま検索すると、一致しない場合があります。

## 検索のスコア

---

検索語が「SQLite 保存」の場合：

| 断片の本文            | 一致する単語数 |
|------------------|---------|
| SQLiteにデータを保存します | 2       |
| SQLiteを使います      | 1       |
| CSSで色を指定します      | 0       |

同じ単語が何回出ても、この実装では1点です。  
0点は除き、点数の高い順に並べます。

## forとifで数える

---

```
score = 0
for word in words:
    if word in text:
        score += 1
```

`words` は空白で分けた検索語のリスト。

`text` は資料名・見出し・本文をつないだ文字列です。

`lower()` で英字の大文字小文字の差を小さくします。



## 上位件数top\_k

---

```
results.sort(key=get_score, reverse=True)  
return results[:top_k]
```

`get_score()` は辞書からスコアを取り出す関数です。

`reverse=True` で大きい順に並べます。

`[:top_k]` は先頭から指定件数まで取り出す書き方です。

## 検索結果をLLMへ渡す

---

```
chunks = search_keyword(request.search_text, request.top_k)
result = ask_with_context(question, chunks)
```

第4回の `all_chunks()` が、検索関数へ変わりました。  
プロンプトを作る関数と、画面の出典表示はそのまま使います。  
変更箇所が少ない理由を説明してみましょう。

## 実習1：検索スコアの穴埋め

---

```
cd session05/exercise  
python init_db.py  
python main.py
```

knowledge.py の TODO を完成させます。  
コメントを外し、仮の raise を削除します。  
検索語「SQLite 保存」、上位件数3で質問します。

## 実習2：上位件数の比較

---

同じ質問と検索語で、上位件数だけを1、3、5に変えます。

- 必要な資料は選ばれたか
- 無関係な資料が増えたか
- 根拠が分散した質問に答えられたか
- APIモードで入力トークンがどう変わったか

一度に変える条件は1つにします。

## 検索が失敗した場合

---

「データが消えない仕組み」を探したいのに、  
資料には「永続化」としか書かれていない場合があります。

キーワード検索は、表記の違いや言い換えが苦手なことがあります。

「SQLite」「保存」などの語を足すとどう変わるでしょうか。

この失敗を、次回の意味検索と比較します。

## 検索と生成の失敗を分ける

---

| 検索結果     | 回答     | 最初に調べる場所     |
|----------|--------|--------------|
| 必要な根拠がない | 誤答     | 検索語・分割・件数    |
| 必要な根拠がある | 誤答     | 指示・文脈・モデルの出力 |
| 根拠がない    | 回答を控える | 検索の改善が必要か    |
| 根拠がある    | 正答     | 別の質問でも確かめる   |

検索結果を見ずに、プロンプトだけを変え続けないようにします。

## 実習3：見つからない質問

---

検索語を、どの資料にもない `zzzz-no-match` にします。

教材では検索結果が0件なら、LLMを呼ばずに  
「参照できる資料が見つかりません」と返します。

0件でない場合にも、答えが含まれるとは限りません。  
その場合の回答保留は、モデルの出力を評価します。

## 資料の鮮度と出典

---

- 原本を編集したらDBを更新する
- 古い案内と新しい案内を混在させない
- 複数の資料が矛盾するときの方針を決める
- 日付が重要な題材では、更新日などのカラムを追加する

教材のIDは再登録で変わる可能性があります。

評価結果には原本ファイル名・見出しも記録します。



## 確認問題

---

1. RAGでは、モデル自体を学習し直している？
2. 上位5件にすれば、必ず上位1件より正確になる？
3. 検索結果0件なら、必ずLLMを呼ぶ必要がある？
4. キーワード検索が見逃しやすい質問は？

## 提出物と次回

---

提出： `knowledge.py` と `worksheet.md` のGitHub URL。

- 検索語と上位件数
- 選ばれた資料のファイル名・見出し・スコア
- 見つかった例、見つからなかった例
- 検索と生成のどちらを直すべきか

次回は、文章をベクトルにして検索します。